

Branch and bound algorithm

Using the template simplex routine, implement the branch and bound algorithm to find the optimal solution and objective for an ILP. Assume all the variables are integers and the problem has only inequality constraint. The slack variables introduced to transform the LP to standard form need not be integers.

- Simplex:

```
import numpy as np
def print_tableau(A, b, c, f):
    for i in range(len(A)):
        print("{0:30} | {1}".format(str(A[i]), b[i]))
    print('-----')
    print("{0:30} | {1}\n".format(str(c), f))

def simplex(m, obj):
    varMap = {v.name:i for i,v in enumerate(m.vars)}
    numVars = len(m.constrs) + len(m.vars)
    A = np.zeros(shape=(len(m.constrs), numVars))
    c = np.zeros(numVars)
    for i, v in obj.expr.items():
        c[varMap[i.name]] = v
    b = np.zeros(len(m.constrs))
    for i, e in enumerate(m.constrs):
        sense = e.expr.sense
        b[i] = e.rhs if sense == '<' else -e.rhs
        for ind, val in e.expr.expr.items():
            A[i][varMap[ind.name]] = val if sense == '<' else -val
        if sense == '<' or sense == '>': A[i][len(m.vars) + i] = 1
    f = 0
    sol = np.zeros(numVars)

    print("Initial tableau:")
    print_tableau(A, b, c, f)

    for i in range(len(m.vars), numVars):
        sol[i] = b[i - len(m.vars)]

    while np.any(c > 0):
        if not np.any(c > 0): break
        pivot = np.argmax(c > 0)
        pr = np.argmin([b[i]/A[i][pivot] if A[i][pivot] > 0 else np.inf for i in range(len(b))])

        scale = A[pr][pivot]
        A[pr] /= scale
        b[pr] /= scale
        for i in range(len(b)):
            if i == pr: continue
            scale = A[i][pivot]
            A[i] -= (scale * A[pr])
            b[i] -= (scale * b[pr])

        scale = c[pivot]
        c -= scale * A[pr]
        f -= scale * b[pr]
        sol = np.zeros(numVars)
        for j in range(numVars):
            if np.sum(A, axis=0)[j] == 1.0:
                i = np.argmax(A[:, j] > 0)
                sol[j] = b[i]
        if verbose: print(f'after : A = {A} b = {b}, c = {c}, f = {f}, sol = {sol}')

    print("Final tableau:")
    print_tableau(A, b, c, f)

    return sol, -f
```

- Template code for submission:

```

from mip import Model, maximize, INTEGER

eps = 1e-6
# define a value as integer if it is within eps of its rounded value
# Nvar are the number of original variables
# The slack variables need not be integers and hence don't need to be checked
def is_sol_integer(sol, Nvar):
    for i in range(Nvar):
        if abs(round(sol[i]) - sol[i]) > eps: return False
    return True

def solve_ilp(m, obj):
    sol, f = simplex(m, obj)
    Nvar = len(m.vars)
    # implement the branch and bound algorithm
    # assume all variables are integers
    return sol, f

```

- Test 1:

```

m = Model()
x = [m.add_var(var_type=INTEGER) for i in range(2)]
obj = m.objective = maximize(x[0] + x[1])
m += x[0] + 3 * x[1] <= 9.2
m += 2 * x[0] + x[1] <= 8.4
sol, f = solve_ilp(m, obj)
solve_ilp(m, obj)
print('solution :', sol[0:len(m.vars)], f'objective : {f}\n')
m.optimize()
print('mip sol :', [v.x for v in m.vars], 'objective :', m.objective.x)

```

- Test 2:

```

m = Model()
x = [m.add_var(var_type=INTEGER) for i in range(3)]
obj = m.objective = maximize(6 * x[0] + x[1])
m += 9 * x[0] + x[1] + x[2] <= 18.4
m += 24 * x[0] + x[1] + 4 * x[2] <= 42.3
m += 12 * x[0] + 3 * x[1] + 4 * x[2] <= 96.5
sol, f = solve_ilp(m, obj)
print('solution :', sol[0:len(m.vars)], f'objective : {f}\n')
m.optimize()
print('mip sol :', [v.x for v in m.vars], 'objective :', m.objective.x)

```